

ICLR 2022: For Offline RL, a Two-Layer MLP Is All You Need

Researchers at UC Berkeley and CMU strip "offline RL via supervised learning" down to its essentials, and find that only model capacity and the choice of conditioning variable really matter

Ask most people how to do offline reinforcement learning, learning a policy from a fixed batch of logged data without ever interacting with the environment, and the answer usually starts with value functions: fit a Q-function, then improve the policy against it. Methods built on temporal-difference (TD) learning are powerful in theory, but they are notoriously fiddly in practice, leaning on stabilization tricks and careful hyperparameter tuning to keep from diverging.

An appealing alternative reframes the RL problem as conditional imitation learning. The key insight is that a trajectory that looks terrible for one goal may be an optimal demonstration for another. If you feed the policy an extra input, the outcome you want it to reach, and then simply fit the logged actions with ordinary supervised learning, you can sidestep value functions entirely. But the recipe had been murky. Different papers reached contradictory conclusions about what makes it work: some insisted on advantage weighting, others reached for large Transformer sequence models. Which components are actually essential, and which are just incidental complexity that a practitioner can safely drop?

This ICLR 2022 paper, from UC Berkeley and Carnegie Mellon, does not propose yet another algorithm. Instead it gathers these methods under one framework it calls RvS (Reinforcement Learning via Supervised Learning) and then, through extensive experiments, boils them down to the essentials. The finding is surprising: a plain two-layer feedforward network (an MLP), trained simply to maximize likelihood, matches the state of the art set by far more complex methods built on TD learning or Transformer sequence models. Only two things turn out to decide the outcome: how much capacity the model has, and what you condition on.

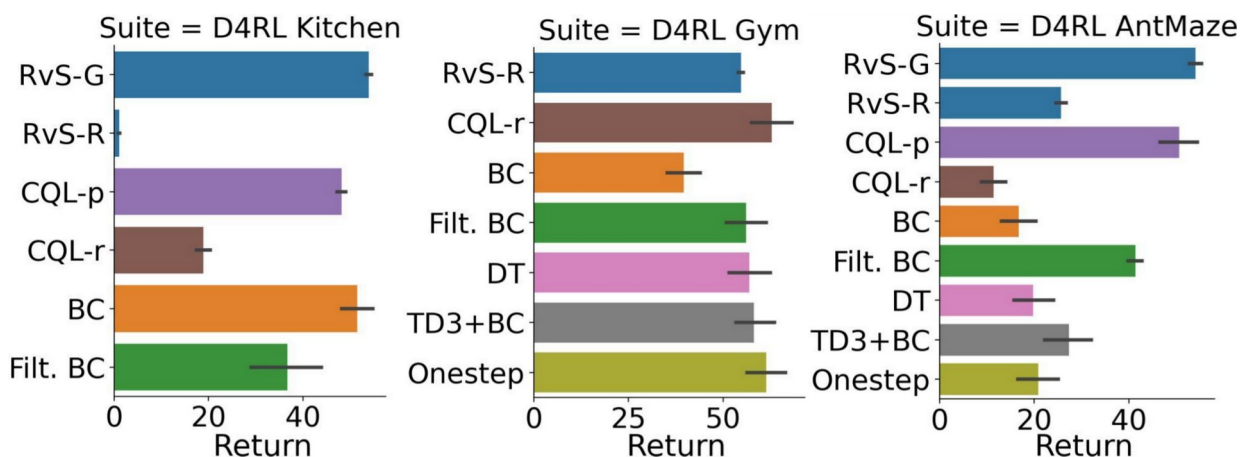


Figure 1. Across the D4RL Kitchen, Gym, and AntMaze suites, goal-conditioned RvS-G and reward-conditioned RvS-R are compared against prior methods and baselines. Each bar is the average return over

many tasks in a suite. Using nothing but supervised learning with a feedforward MLP, RvS matches methods that rely on TD learning and Transformer sequence models.

Paper: <https://arxiv.org/abs/2112.10751> Code: <https://github.com/scottemmons/rvs>

Why bother stripping the method down

Value-based methods dominate offline and off-policy RL, and they come with attractive theoretical guarantees. In practice, though, they are hard to deploy: they require a stack of tricks to stabilize training and delicate tuning across many hyperparameters. Recasting RL as a conditional, filtered, or weighted imitation-learning problem is tempting precisely because it only asks for supervised learning. If a minimal supervised recipe can match the complex methods, it gives practitioners a dependable field guide, and it also reveals exactly where such methods still break down.

The paper's contributions come in three parts. First, rather than inventing a new algorithm, it places existing goal-conditioned and reward-conditioned methods under the single RvS framework. Second, through large-scale experiments it distills them to their essence, showing that a two-layer MLP trained with maximum likelihood is competitive with much more complex, state-of-the-art methods. Third, it pins down which choices genuinely matter (capacity, regularization, and the conditioning variable) and is honest about the limits: on purely random data, RvS is comparatively weak, a boundary the authors are careful to flag.

RvS: a deliberately simple framework

The setup is plain. An agent lives in a Markov decision process, and we train a policy conditioned on an outcome, $\pi(a \mid s, \omega)$, where the outcome ω can be a future goal state (giving RvS-G) or the average return over some future window (giving RvS-R). Given a batch of offline trajectories, RvS applies hindsight relabeling to build its training set: every action that actually occurred becomes a demonstration for whatever outcome that trajectory actually reached.

It helps to separate the two flavors of outcome. In RvS-G the outcome is a goal state, so the policy learns to reach configurations of the world; in RvS-R it is an average future return, so the policy learns to hit a level of performance. Hindsight relabeling is what makes both trainable from unlabeled logs: because we read off the outcome each action actually led to, even aimless or suboptimal experience becomes a valid demonstration for some outcome. That is the same insight that lets a trajectory which failed at one task serve as a perfectly good example for another.

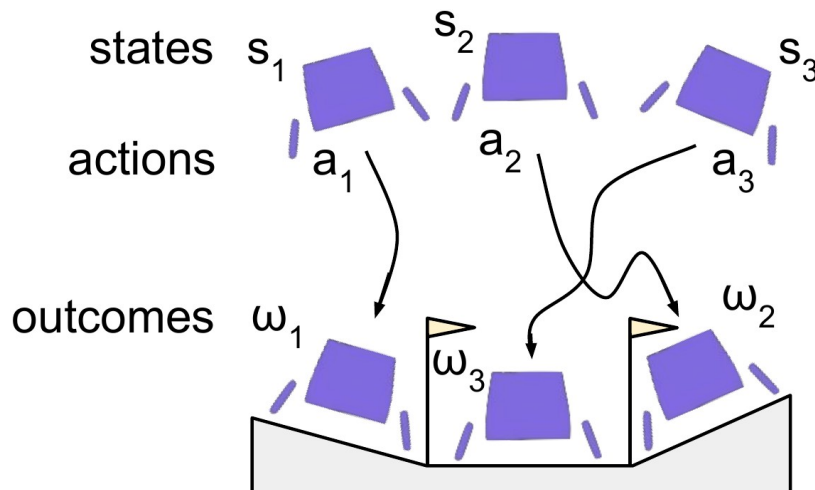


Figure 2. Hindsight relabeling. States s and actions a from a trajectory are relabeled as demonstrations toward the outcomes ω they actually reached, turning a batch of unfiltered experience into supervised (state + goal $\rightarrow$ action) data.

With that dataset in hand, the training objective is nothing more than maximizing the log-likelihood of the observed actions under the conditioned policy: concatenate the outcome ω onto the input state s , and let a two-layer fully connected MLP predict the action. There is no advantage weighting, no TD bootstrapping, and no Transformer, just a maximum-likelihood objective over relabeled experience. At test time you hand the policy the outcome you want (a goal state or a target return) and let it act.

Where it is evaluated

The evaluation is broad, spanning navigation, locomotion, and manipulation. On the D4RL benchmark the authors use three suites: AntMaze (an 8-DoF quadruped navigating a maze), Gym Locomotion (HalfCheetah, Hopper, and Walker, with data quality ranging from random to medium-expert), and Franka Kitchen (a 9-DoF manipulation task built from human demonstrations). They add the GCSL suite of goal-conditioned tasks, including 2D navigation, a Sawyer arm, Lunar Lander, and a robotic claw, adapted to the offline setting with data collected by a random policy. All scores are normalized to the $[0, 100]$ range for comparison.

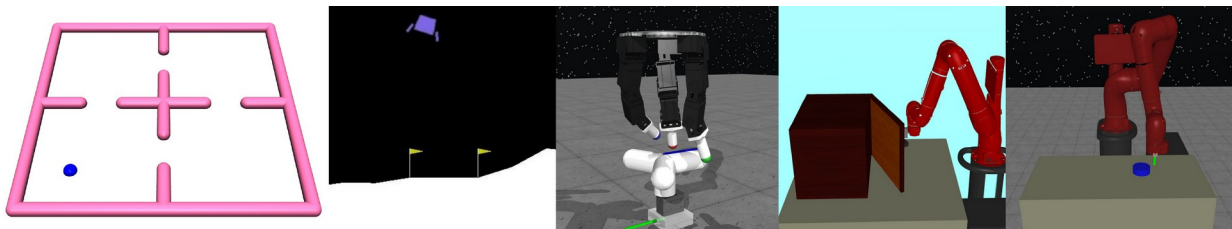


Figure 3. Several goal-conditioned environments from the GCSL suite (left to right: FourRooms navigation, Lunar Lander, the robotic Claw, Door opening, and Pusher). Together with the D4RL locomotion and manipulation tasks, they form a broad sweep from navigation to fine-grained control.

The result: an MLP reaches state of the art

The headline is striking. Using only a two-layer feedforward network and a maximum-likelihood objective, RvS reaches state-of-the-art performance across several suites. On AntMaze, goal-conditioned RvS-G averages 53.5, edging out the strongest value-based baseline, CQL-p at 50.6. On Kitchen it reaches 54.0. On the GCSL suite it scores 62.0, beating online GCSL at 58.0, and it does so from purely offline, random-policy data. On Gym Locomotion, reward-conditioned RvS-R matches Decision Transformer while using a simple MLP rather than a large Transformer.

The most counterintuitive part concerns subtrajectory stitching. The mixed dataset in Kitchen and all of the AntMaze tasks require assembling fragments of different trajectories, and the field long assumed such tasks demanded dynamic-programming-style value methods. RvS-G keeps pace here too, which suggests that supervised learning is not as inherently incapable of stitching as one might expect. To be fair about the picture: on Gym, value-based methods (such as CQL-r at 63.0) still score higher, so RvS is best read as matching the state of the art with the simplest possible means, not as beating everything outright.

Table 1. Normalized average scores per suite (excerpted from the paper's Table 1; scores in [0, 100]).

Suite	BC	Decision Transformer	Best value-based	RvS-R	RvS-G
D4RL AntMaze	16.7	19.8	50.6 (CQL-p)	25.6	53.5
D4RL Gym	39.6	57.0	63.0 (CQL-r)	54.7	—
D4RL Kitchen	51.5	—	48.2 (CQL-p)	1.0	54.0
GCSL	—	—	58.0 (online GCSL)	—	62.0

Note: the 58.0 in the GCSL row is the online GCSL method. Some baselines marked with * in the paper use AntMaze v0; see Appendix A of the original paper. RvS-G is not run on Gym, which is framed as reward maximization, so that cell is left blank.

What actually matters: capacity and regularization

Because the model is so simple, its success rests on a handful of design choices, and the ablations nail them down one by one. The first is capacity: the best architectures are larger than those used in standard online RL or imitation learning, and widening the hidden layers up to 1024 units generally keeps helping. The second is regularization: dropout is not universally good. It boosts performance on the small, human-demonstration kitchen-complete dataset, but it hurts on hopper-medium-expert and antmaze-medium-play.

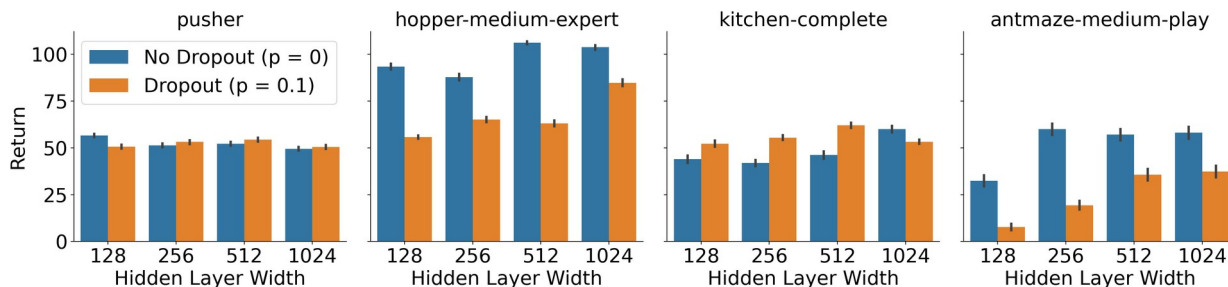


Figure 4. Capacity and regularization ablation across four environments, varying network width (x-axis) against dropout (blue = none, orange = $p=0.1$). Larger networks help on hopper-medium-expert and kitchen-complete,

underscoring the value of high-capacity policy networks, while dropout is a reliable win only on small datasets like kitchen-complete.

One reading is that kitchen-complete is small and drawn from human demonstrations, so it is prone to overfitting and benefits from the extra regularization dropout provides, whereas on tasks with more data (or data that is already close to random) forcing dropout only throttles the policy's capacity. In other words, high-capacity policy networks and just-enough regularization have to be tuned together to get strong results.

The third choice is the policy's output distribution. The paper compares a categorical distribution over a discretized action space against a unimodal Gaussian over continuous actions. Across the GCSL suite, the categorical distribution generally matches or beats the Gaussian, echoing the same theme that more policy capacity helps. There is also a useful caution for anyone tuning these models: validation loss correlates only loosely with final performance, so it is not a reliable signal to tune against in practice.

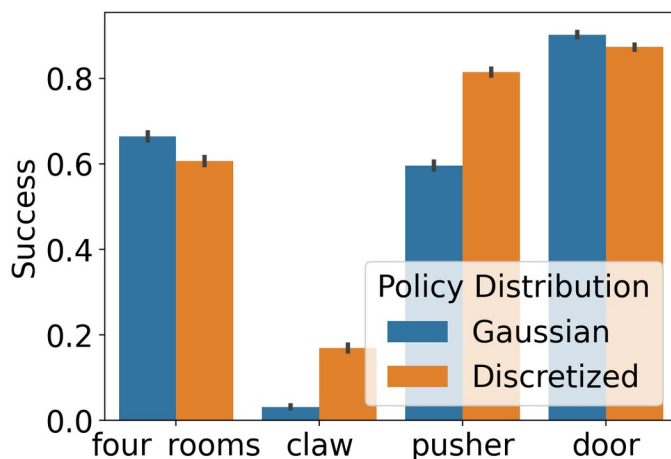


Figure 5. Across four GCSL environments, a categorical distribution over discretized actions (orange) generally matches or exceeds a unimodal Gaussian over continuous actions (blue), consistent with the broader pattern that higher-capacity RvS models perform better.

A field guide you can use today

Put together, the paper hands practitioners a field guide. To do offline RL by supervised learning, you do not need advantage weighting and you do not need a Transformer. A two-layer feedforward network trained simply to maximize likelihood is competitive with the state of the art, as long as you get two things right: tune the model's capacity and regularization carefully (grow the width until performance saturates, then add a little dropout if the dataset is small), and choose the right conditioning variable, trading off goals against rewards depending on the task.

The authors do not dodge the boundary of their result. On purely random data, TD learning still wins and RvS is clearly weaker, which they flag as an open problem for future work. For a reader, the value of the conclusion lies in that restraint: it does not claim supervised learning has solved offline RL, but tells you clearly when the simplest method is already good enough, and where it will fall short.